



BUILD A WORLD MODEL FROM SCRATCH · LECTURE 2

The World Modeler's *Toolkit*

Four tools, built carefully: compressing the world into a few numbers, then reward over time, value, and the two ways to learn behaviour.

LATENT SPACE

REWARD OVER TIME

VALUE

TWO WAYS TO ACT

Vizuara AI Labs · Build a World Model from Scratch

Last lecture: the loop. This lecture: the tools.

Lecture 1 gave us the picture: an agent and an environment, trading actions for observations — and the state hiding behind what we see.

● Next lecture, we build

Our first learned world model, in code, from an empty file.

● But four tools come first

Every paper in this field assumes you know them. Most never explain them. Today we build each one from zero — no RL background assumed.

● Nothing here is busywork

Each tool answers a question the loop forces on us. You'll see the question first, the tool second.



Same loop as last time — now we ask: how does an agent get *good* at it?

Four tools — that's the whole lecture

1 · Compression — the latent space

Squeezing an image into a handful of numbers, and why every world model lives there.

2 · Reward over time

Good decisions pay off *later*. The return, the discount, and the credit-assignment problem.

3 · Value

One number that says how good a situation is — counting all the reward still to come.

4 · Two ways to learn behaviour

Score every action and pick the best — or keep an actor and a critic. Why world models chose the second.

Explanation from Rajat

— WHITEBOARD —



TOOL 1 OF 4

Compression: The Latent Space

The first tool is about perception: turning a hundred thousand pixels into a handful of numbers that keep only what matters.

ENCODER · BOTTLENECK · DECODER

The sketch-artist game



A million pixels, across the table in twenty words.

A witness describes a face to a sketch artist — in maybe **twenty words**. The artist draws. The drawing comes out remarkably close.

- **Think about what just happened**

A face is a million pixels. It crossed the table as twenty words — and survived. The description kept *what mattered* and dropped everything else.

- **The witness is an encoder**

Face → compact description.

- **The artist is a decoder**

Compact description → face again. And the quality of the redrawing tells you how good the description was.

Now replace the words with numbers, and both people with neural networks.

The autoencoder: squeeze, then reconstruct

1 Encoder squeezes

The image is forced through a narrowing network into a tiny vector — say **32 numbers**. That vector is the **latent**.

2 Decoder redraws

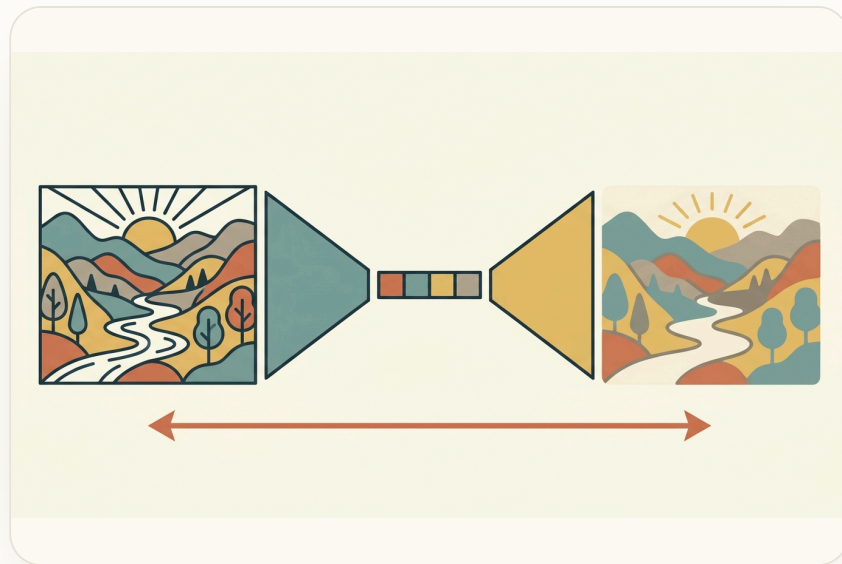
From those 32 numbers alone, a second network tries to redraw the original image.

3 The error is the teacher

Compare redrawing to original, pixel by pixel. Every mismatch flows back as a lesson: *keep more of what I lost*. No labels needed — the image grades itself.

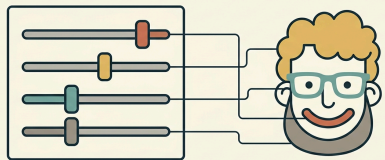
The bottleneck is the whole trick

Make the middle narrow enough and the network has no choice but to discover what's essential. Compression isn't a compromise — it's *how the understanding happens*.



Wide → narrow → wide. The narrow middle is where the meaning lives.

Those 32 numbers form a *space*



Each latent number: a slider on the image.



Similar images live near each other.

- **Each number tends to mean something**

Turn one up and the smile widens; another, and the lighting shifts. The latent is a small control panel for the image.

- **Nearby points are similar images**

Two faces that look alike land close together. Similarity becomes *distance* — something you can compute with.

- **And that's what makes prediction possible**

"Predict the next frame" becomes "move to a nearby point" — a small, smooth step in a small, smooth space, instead of repainting a hundred thousand pixels.

Why every world model lives in latent space

Remember the plan: a world model must predict the future **over and over, thousands of times**, to dream a single episode.

● In pixels: hopeless

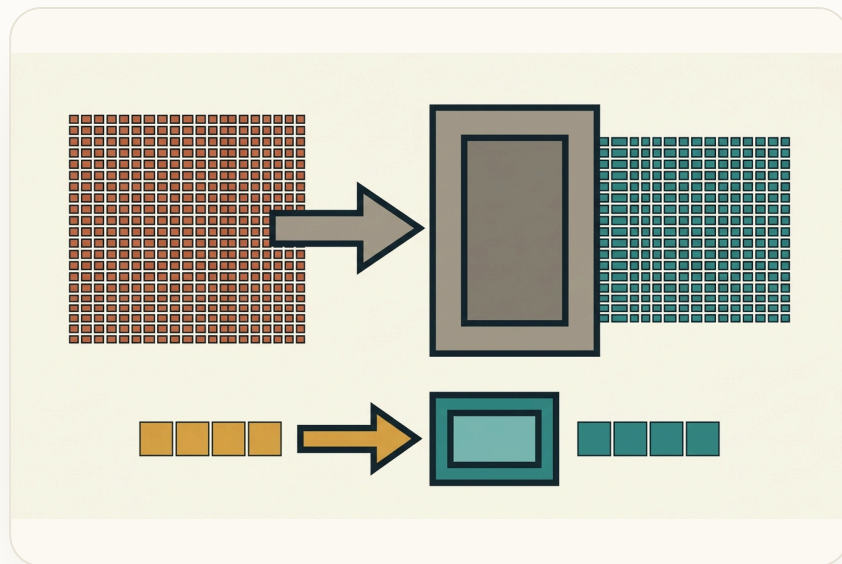
A 64×64 colour frame is 12,288 numbers. Predicting all of them, every imagined step, for a thousand steps — most of it wasted on textures no decision depends on.

● In latents: routine

Encode once → predict 32 numbers per step → decode only when you want to look. Hundreds of times cheaper, and the model spends its capacity on what actually changes the future.

● This is the recipe of the whole series

Compress, then predict. Every system we build — from next lecture to Dreamer 4 — is this one sentence, refined.



Same prediction job, two sizes. The small one wins everywhere.

Explanation from Rajat

— WHITEBOARD —



TOOL 2 OF 4

Reward Over Time

The reward you get right now is almost never the point. Good behaviour is about what comes after.

RETURN · DISCOUNT · CREDIT ASSIGNMENT

The problem: the best move often looks bad right now

● Chess: the sacrifice

Giving up your queen is an immediate loss — and sometimes it's the move that wins the game five moves later.

● Life: the exam

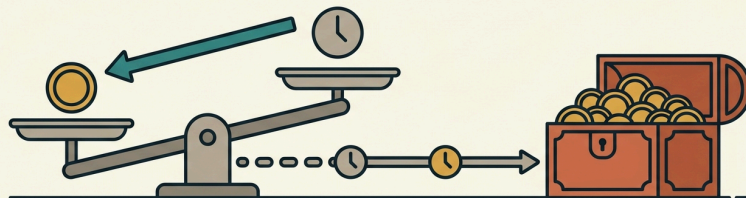
Studying tonight costs you an evening. The payoff arrives months away.

● So: never judge an action by its instant reward

Judge it by the **total** reward it leads to — everything from now until the end. That total has a name: the **return**.

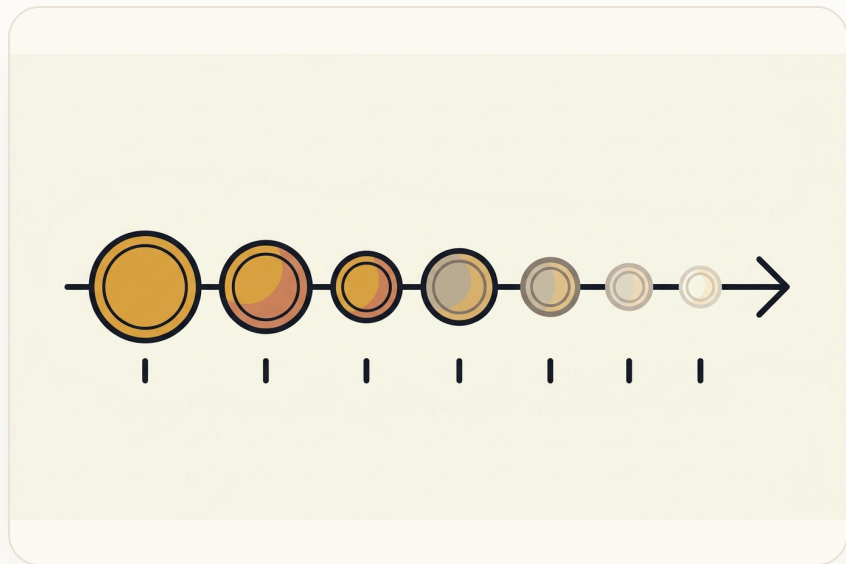
$$\text{return} = r_{\text{now}} + r_{\text{next}} + r_{\text{after}} + \dots$$

the number an agent actually tries to maximise



One coin now, or a chest later? Behaviour is choosing between *streams* of reward.

The discount: tomorrow's coin is worth slightly less



Same coin, fading with distance. That fade is γ , γ^2 , γ^3 ...

We shrink each future reward by a factor γ (gamma) per step — say 0.99.

$$\text{return} = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2 + \gamma^3 \cdot r_3 + \dots$$

γ between 0 and 1 • the further away, the smaller the weight

Why it's honest

The far future is uncertain — the game may end, the world may change. With $\gamma = 0.99$, a reward 100 steps away counts ~37% of one today.

Why it's practical

Without it, an endless game gives an infinite sum. Discounting keeps the total finite.

One dial, one meaning

γ near 0: greedy, lives for today. γ near 1: patient, plans far ahead.

The hard part: who deserves the credit?

You win the game at move 40. Which of your forty moves *caused* it?

- **The reward arrives late and says nothing**

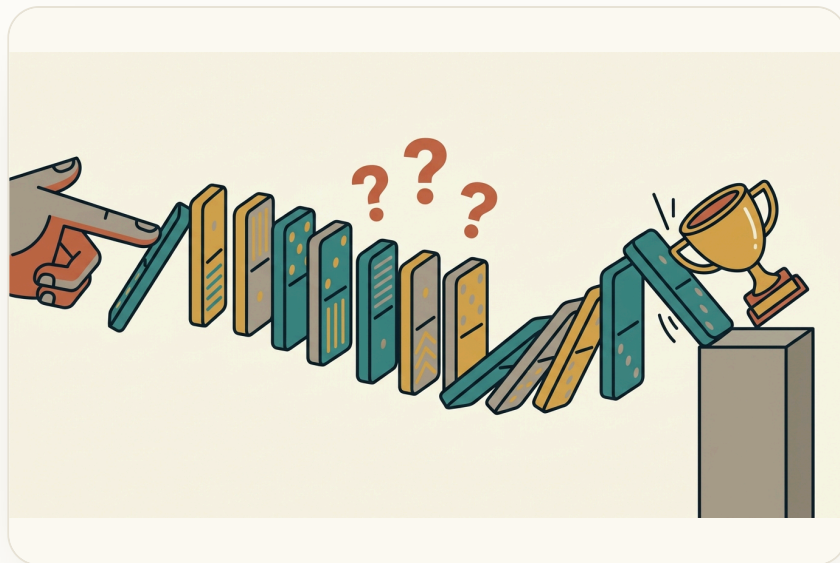
One "+1, you won" at the end. Silence before that. The signal doesn't point at the brilliant move 12 — or the blunder at move 30 that almost lost it.

- **This is the credit-assignment problem**

Spreading one late reward correctly across the many decisions that led to it. It is *the* central difficulty of learning behaviour.

- **And the reason the next tool exists**

We solve it by learning a number that fills the silence — a running estimate of how well things are going at *every* step, not just the last one.



The last domino gets the trophy. Which push deserves the credit?

Explanation from Rajat

— WHITEBOARD —



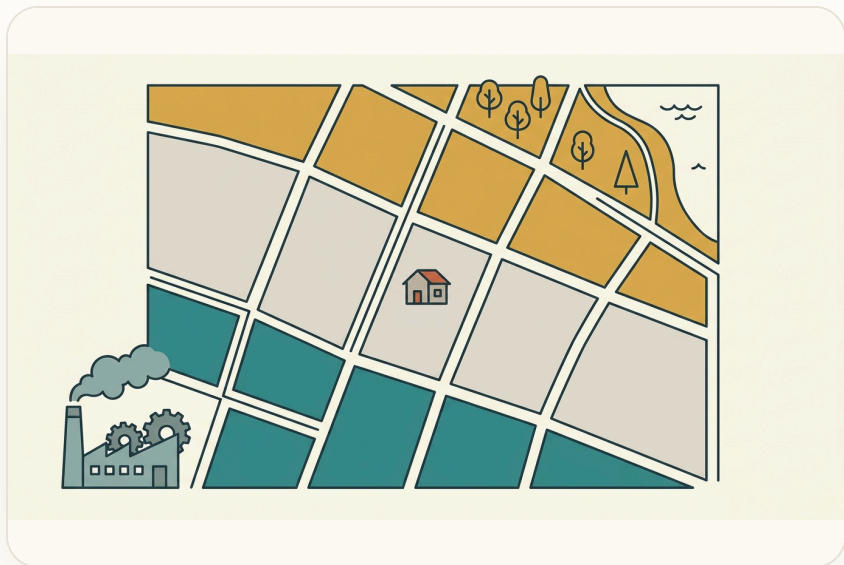
TOOL 3 OF 4

Value

One number per situation: how much total reward should I expect from here? The tool that fills the silence between rewards.

$V(S) \cdot Q(S, A)$

Value is a price tag on a situation



A value function is a price map over situations: warm near the good future, cool near the bad one.

Why does a house near the park cost more than the same house near the factory? Not because of today — because of **everything living there will bring, added up.**

$V(s)$ = expected return, starting
from s

the value function — a learned price tag for every state

● Same idea in chess

A grandmaster looks at a position and says "White is winning" — long before any piece is taken. That judgement *is* a value estimate.

● And it fills the silence

Reward speaks once, at the end. Value speaks at *every step*: did this move raise my prospects or lower them? That's how one late reward becomes forty per-move signals — credit assignment, solved.

Two flavours: value of a place, value of a choice

$$V(s)$$

"how good is it to be here?"

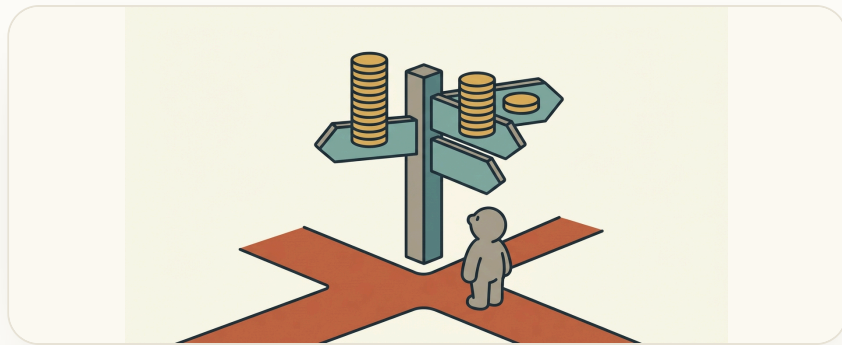
One number for the situation itself. The house's market price. The grandmaster's "White is winning."

Keep both letters in your pocket. V prices situations; Q prices choices. Every algorithm in this series is built from one of these two numbers — and you now know both.

$$Q(s, a)$$

"how good is it to do *this*, here?"

One number per *action*: from this fork, what does each road lead to? Know Q for every action, and acting is trivial — take the biggest number.



Why world models especially need value: seeing past the horizon

A world model can imagine the future — but only so far before the dream gets unreliable. Say 15 steps. **What about everything after step 15?**

- **Value is the telescope**

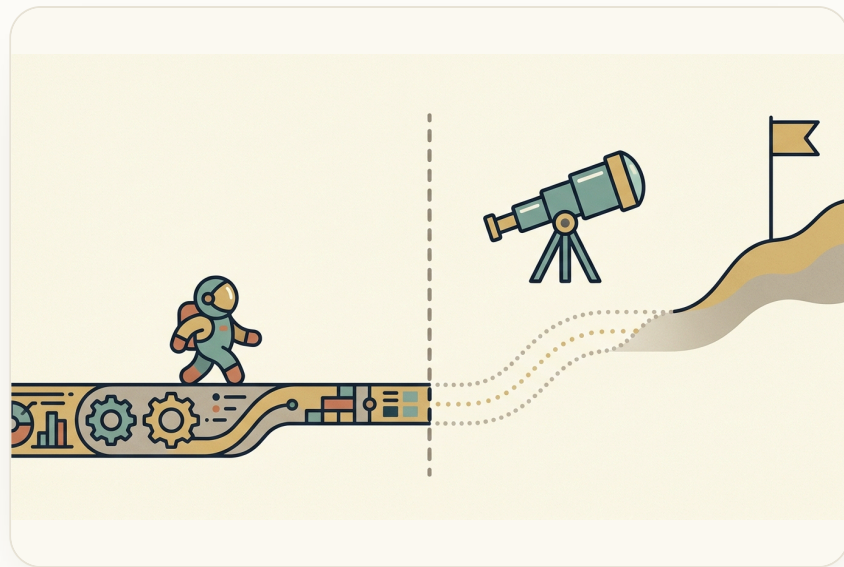
Imagine 15 steps in detail, then let $V(s_{15})$ summarise *all the rest* in one number. Detailed dream up close; a single learned estimate beyond.

- **Every dreaming agent works this way**

Short imagined rollout + value at the end. It's how an agent with a 15-step imagination still plays for outcomes hundreds of steps away.

- **Remember this picture**

When we build imagination-trained agents later in the series, this slide is the reason the critic exists.



Walk the first stretch in detail; let one number stand in for the rest.

Explanation from Rajat

— WHITEBOARD —



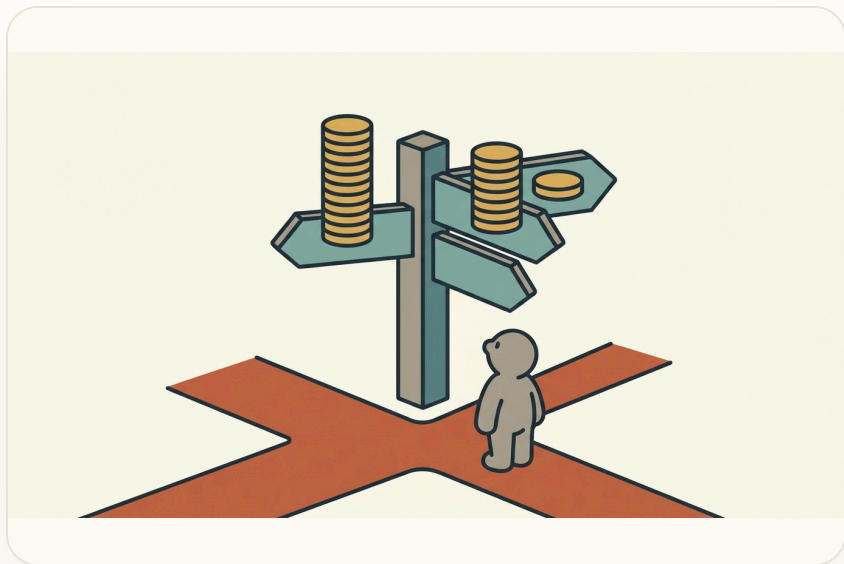
TOOL 4 OF 4

Two Ways to Learn Behaviour

With reward and value in hand, how does an agent actually decide what to do? The field settled on two answers.

VALUE-BASED · ACTOR-CRITIC

Way 1 – score every action, take the best



Every action wears its price. Behaviour = take the tallest stack.

Learn $Q(s, a)$ for every action, then behave greedily: **always pick the action with the biggest number.** That's Q-learning — the family behind DQN and the 2013 Atari breakthrough.

● Where it shines

A handful of discrete choices — left, right, jump. Score each, compare, done.

● Where it breaks

Continuous actions. A robot arm's motor command is a list of real numbers — infinitely many candidates. You cannot score them all and take a max.

Notice there's no "policy" stored anywhere — behaviour is just *read off* the score table, fresh, every step.

Way 2 – keep an actor, and a critic

Split the job into two learners:

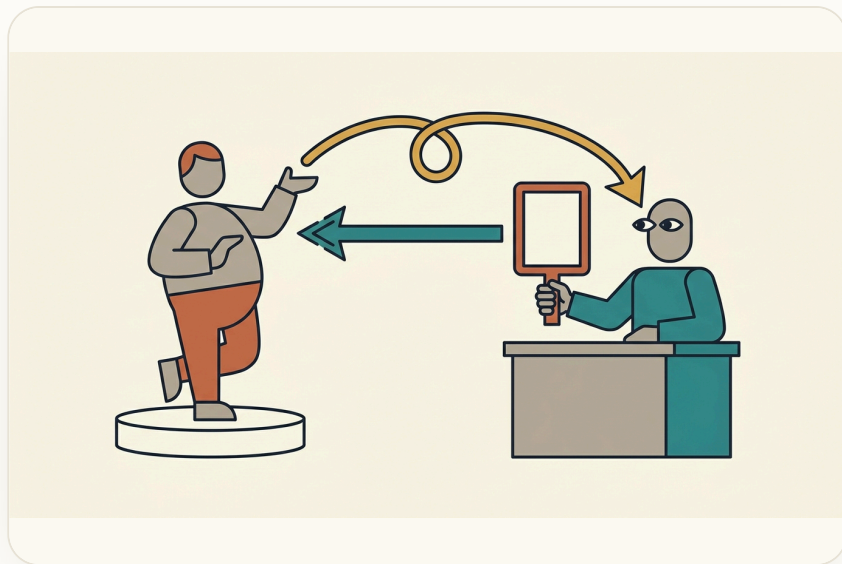
The actor — a policy

A network that directly outputs the action: see the situation, propose the move. No table, no max — it can output continuous values as easily as button presses.

The critic — a value function

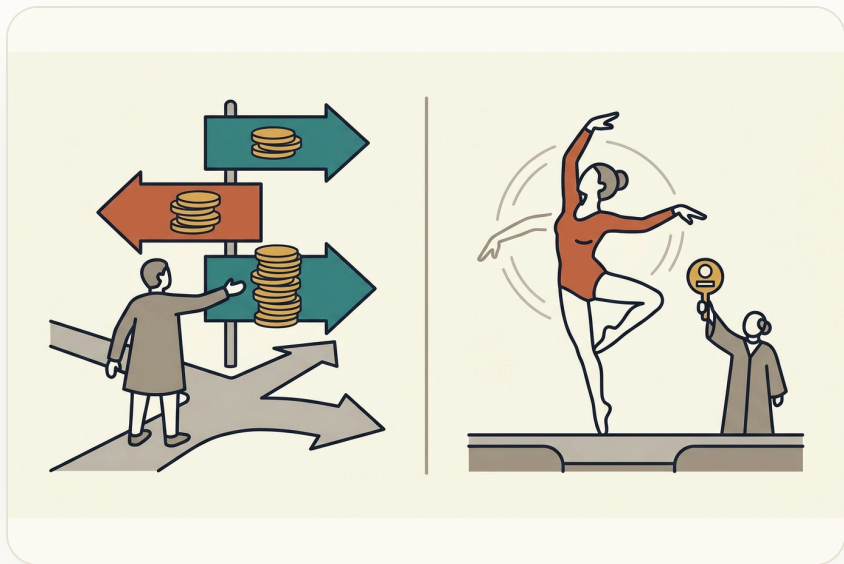
A second network that watches and scores: "that choice raised your prospects" / "that one lowered them." The critic's judgement is the actor's training signal.

Rehearse a move, get scored, adjust, repeat — a performer and a coach. The actor slowly bends toward whatever the critic rates highly.



The actor performs; the critic scores; the score shapes the next performance.

Why world models chose actor-critic



Two ways to act. World models picked the right one — for reasons, not fashion.

- **Dreams are perfect training grounds for a critic**

Inside imagination the agent can rehearse thousands of episodes cheaply — and the critic can score every imagined step. The pair thrives exactly where a world model lives.

- **Real bodies have continuous actions**

Robot joints, steering angles, torques. The actor outputs them directly; a Q-table cannot.

- **And the critic is the horizon telescope**

Tool 3's picture: short dream + value at the end. The critic *is* that value. Actor, critic, and world model click together like parts of one machine.

This is why Dreamer-family systems — including the one we'll reproduce as our capstone — are all actor-critic inside a dream.

Explanation from Rajat

— WHITEBOARD —

Four tools to keep

1

Compress, then predict

An autoencoder squeezes the frame into a latent; the world model predicts in that small space. The series in one sentence.

2

Judge actions by the return, not the reward

Total future reward, gently discounted by γ . The whole difficulty is that credit arrives late.

3

Value is a price tag on a situation

$V(s)$ for places, $Q(s, a)$ for choices. It fills the silence between rewards — and sees past the imagination horizon.

4

Actor-critic is the world model's partner

A policy that acts, a critic that scores — the pair that trains happily inside a dream, with continuous actions.

Lecture 3 — The First World Model

The toolkit is complete. Next time we use every tool at once — on the **2018 paper that named the field**: Ha & Schmidhuber's "World Models".

- **V · M · C — an eye, an imagination, a hand**

Three small networks, each one built from a tool you now own.

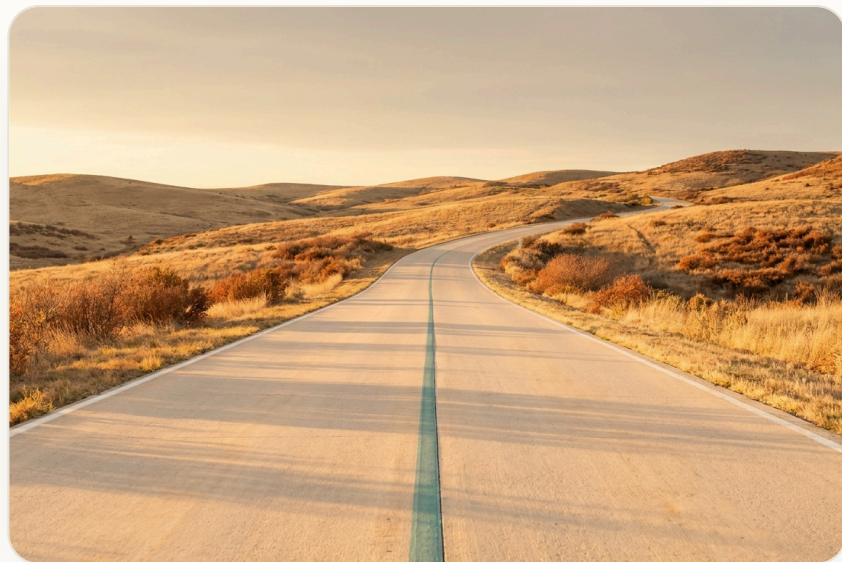
- **An agent trained inside its own dream**

The real game unplugged; every kilometre of practice imagined.

- **And the trick that made it transfer**

Why the policy started cheating the dream — and how the paper stopped it.

●●● Vizura AI · Build a World Model from Scratch



See you in Lecture 3.